06 - Backend Design

Generated 2026-10-14 from 06-BACKEND-DESIGN.md - information only, not medical advice.

Contents

Composition root

Modules

Configuration

Scripts (backend/scripts/)

Location: `backend/` · Python 3.11, FastAPI (sync handlers, one uvicorn worker per container), LangChain + LangGraph, SQLAlchemy/psycopg 3, Redis, pydantic-settings.

Composition root

`app/container.py` builds everything from `Settings` and accepts overrides for every external dependency (model, classifier, retriever, embedder, index, registry, OCR, key resolver, DB probe, KV, conversation store, web search, agent selector). Tests pass fakes; production passes nothing.

Backend choice	When
`pgvector`	`DATABASE_URL` set (`RETRIEVAL_BACKEND=auto`) - production
`memory`	in-process index/registry (tests, demos)
`static`	Stage-1 BM25 over a JSON corpus (tests only)

Modules

Module	Responsibility
`api/deps.py`	Bearer token -> `Principal`; `require_user/clinician/admin`; stores user id on `request.state` for the audit line
`api/errors.py`	`AppError` -> JSON + status (+Retry-After); unhandled -> 500 with traceback logged
`api/routes_*.py`	health, chat/research, SSE + sources + evaluations, documents, conversations (+report), review
`rag/service.py`	The safety pipeline (see LLD 4.1)
`rag/trace.py`	`AgentTrace`: per-agent decisions and timings for "How this answer was produced" (LLD 4.1.4)
`rag/ingest.py`, `loaders.py`, `ocr.py`, `chunking.py`	Ingestion
`rag/retrieval.py`, `rerank.py`, `versions.py`	Retrieval
`rag/cache.py`, `memory.py`, `review.py`, `report.py`, `evaluation.py`	Supporting use cases
`rag/providers.py`	OpenAI-compatible chat/embeddings/tool models, pgvector index, SQL stores, Redis, Tavily, schema init
`safety/*`	Triage, scope, answer checks, templates, safety-suite runner
`guardrails/*`	Policy engine + YAML
`core/*`	Logging (text with `extra=` / JSON), metrics, audit, auth, circuit breaker, rate limiter, KV

Configuration

All settings are environment variables documented in `.env.example` (repo root). Production refuses to start without an explicit `EMERGENCY_NUMBER`, `DATABASE_URL`, JWKS/Supabase URL and `CONVERSATION_ENCRYPTION_KEY`. Answer temperature is bounded to 0.1-0.3; classifiers/judges are fixed at 0.

Scripts (`backend/scripts/`)

`db_init.py` (schema), `ingest_corpus.py` (manifest ingest, `--dry-run` call estimate), `live_check.py` (4 live questions), `make_synthetic_dataset.py` , `ui_demo_server.py` (real app + fake model for UI tests), `prepush_scan.py` (keys + patient-like data).